A dynamic model of resource allocation in multi-project research and development systems

Nelson P. Repenning[a]

Nelson P. Repenning is an assistant professor of management at the MIT Sloan School of Management. His work focuses primarily on understanding the factors that contribute to the successful implementation, execution, and design of business processes. Current research interests include organizational change, process improvement applied to new product design, and the development of cross-disciplinary management theory. His work draws on a number of modeling methods including simulation, non-linear dynamics, and game and contract theory. He is currently working with MIT's Center for Innovation in Product Development, and the Harley-Davidson Motor Company.

Address: E53-339, 30 Wadsworth Street, Cambridge, MA 02142, USA. nelsonr@mit.edu

Abstract

Managers and scholars have increasingly come to recognize the central role that design and engineering play in the overall process of delivering products to the final customer. Although significant progress has been made in the *design* of effective product development processes, many firms still struggle with the *execution* of their desired development process. In this paper a model is developed to study one hypothesis to explain why firms experience such difficulties. The analysis of the model leads to a number of new insights not present in the existing literature. First, the analysis shows that under a plausible set of assumptions product development systems have multiple steady-state modes of execution (or equilibria). This insight suggests that it is possible for product development systems to get ''trapped'' in a state of low performance. Second, the analysis highlights that, for multiple equilibria to exist, a positive loop must dominate the system. The conditions required for such dominance are also provided. Third, the analysis demonstrates that the sensitivity of the system to undesirable self-reinforcing dynamics is determined by the utilization of resources. Fourth, simulation experiments show that testing delays also play a critical role in determining the system's dynamics. Finally, one extension to the model is considered, the introduction of new development tools, and policies for performance improvement are discussed. Copyright © 2000 John Wiley & Sons, Ltd.

Syst. Dyn. Rev. **16**, 173–212, (2000)

Introduction

System dynamics (SD) has a long tradition of using formal models to study research and development (R&D) processes. This focus emerged in the early days of the field and has grown to be both an active area of research and a domain in which SD has significantly influenced management practice. Initially, work covered a range of topics. In 1978 Roberts (1978: 279) wrote

> . . . work [in R&D] can be divided into three areas: (1) the dynamics associated with research and development projects; (2) phenomena associated with the whole of the R&D organization, especially resource allocation among projects or areas; and (3) interrelations between the R&D effort and the total corporation (or government agency).

Since the publication of Roberts's book, however, work in SD has increas-

[a] Correspondence to: Nelson P. Repenning, E53-339, 30 Wadsworth Street, Cambridge, MA, 02142, USA. nelsonr@mit.edu

For more information on the research program that generated this paper, visit our World Wide Web site at http://web.mit.edu/nelsonr/www/.

NSF Cooperative Agreement Number EEC-9529140

System Dynamics Review Vol. 16, No. 3 (Fall 2000): 173–212
Copyright © 2000 John Wiley & Sons, Ltd.
Received May 2000
Accepted September 2000

ingly focused on models of specific development projects and less has been written about multi-project management or the relationship between the R&D function and the rest of the enterprise.

The lack of interest in other approaches to understanding the R&D function may be attributed, in part, to the dramatic success of single-project models. First treated in detail by Roberts (1964), the approach was subsequently used by the Pugh–Roberts consulting company to help the Ingalls shipyard settle a multi-million dollar claim against the United States Navy (Cooper 1980). Since then, Pugh–Roberts has become the industry leader in developing such models for litigation, bidding, and project management.

Similarly, the scholarly literature on single-project models has grown to include applications to software development (Abdel-Hamid and Madnick 1991), the construction of pulp and paper mills (Homer *et al.* 1993), and the design and development of semi-conductors (Ford and Sterman 1998). Measured in terms of dollar savings, single-project models may have influenced management practice more than any other application of the SD method.

While the dynamics surrounding single development projects continue to be an important area of inquiry (see Ford and Sterman 1998 for recent developments), the second element of the research program outlined by Roberts (1978) presents an important and timely opportunity for SD modelers for at least two reasons. First, both product development scholars and practitioners have increasingly come to realize the importance of focusing on the product development process rather than individual projects. For example, a multi-project focus appears in many currently popular frameworks for managing the R&D function:

- Sanderson and Uzimeri (1997) study Sony's development of personal stereo systems and show how Sony has been able to target numerous markets at low incremental cost using a *product family* strategy;
- Wheelwright and Clark (1992, 1995) suggest that senior managers need to focus their attention on their *portfolio* of products rather than individual projects;
- more recently, Cusumano and Nobeoka (1998) discuss how Toyota has dispensed with its famous ''heavy weight project manager'' system in favor of a multi-project approach.

Second, while interest in managing the R&D function (as opposed to specific projects) is on the rise, there are few formal models focused on understanding multi-project development environments. One of the few articles that does contain such a model, Adler *et al.* (1995), highlights the difficulties of tackling this problem using a traditional management science approach. These authors

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

use a queuing formulation in which the product design and engineering function is modeled as a stochastic processing network and ''...engineering resources are workstations and projects are jobs that flow between workstations.'' A core assumption of this approach is that work moves but resources do not. Such an approach is appropriate in manufacturing where machines are typically attached to the shop floor and in some R&D processes, like testing operations, that rely on dedicated people and machinery. Other resources like design engineers, however, are more mobile and fungible. In some cases engineers follow the projects they work on through multiple phases, while in others engineers work on multiple projects in different phases of their development cycles. In contrast to the queuing-based approach, the SD method, with its emphasis on operational representation, can comfortably handle the movement of resources. In fact the mobility of resources in R&D environments is recognized in the one article in the SD literature (Weil *et al.* 1978) that deals with a multi-project setting. Thus, the dynamics surrounding the management of multiple R&D projects are:

1. increasingly of interest to both product development scholars and practitioners;
2. a research area in which the SD method is uniquely positioned to make important contributions.

To that end, in this article I hope to accomplish four objectives:

1. re-establish models of multi-project development environments as an open, active, and important line of research in SD;
2. present a specific model of a multi-project development system with interesting implications for both research and practice;
3. present a novel method of model analysis;
4. connect two disparate strands in SD modeling: R&D models and work on the dynamics of successful process improvement (e.g., Sterman *et al.* 1997; Repenning and Sterman 2000; Repenning 1999b).

The article is organized as follows: in the remainder of this section I outline the problem towards which the modeling effort is directed and discuss my main hypothesis. In section two the basic model is developed and analyzed. In section three the possibility of improving the process through the use of new tools is introduced. Section four contains discussion and concluding thoughts.

The problem: why don't people use the tools?

Managers and scholars have increasingly come to recognize the central role that design and engineering play in the overall process of delivering products

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

to the final customer (Dertouszos *et al.* 1989), and a variety of authors suggest ways to develop new products faster, cheaper, and with higher quality (e.g., Cooper 1993; Wheelwright and Clark 1992, 1995; Ulrich and Eppinger 1995; Zangwill 1993). Yet, despite the recent advances in process *design*, many organizations struggle with the *execution* of their desired product development process. For example, a manager at one of the companies that provided the basis for this study describes his efforts to induce the engineering staff to properly execute a new development process that required extensive use of computer-aided design and development tools (see Repenning 1996):

> We . . . tried everything—computer rooms, terminals on desks, each engineer with his or her own PC—and at the end of seven years we had seen very little progress except for in the drafting room . . . the question we struggle with was, why couldn't we get the engineers to use the tools?

Similarly, another engineer in the same organization describes his experience with the effort:

> To be perfectly honest, I really don't think [the new process] changed the way engineers did their jobs. In many ways we worked around the system. Good, bad, or indifferent that's what happened. We had a due date and we did whatever it took to hit it.

Paradoxically, the difficulty this organization experienced in executing its desired development processes cannot be attributed to participants' beliefs about its efficacy. In fact, extensive interview data suggest almost universal agreement concerning the process's benefits. One engineer summed up his experience by saying "... the basic tenets of [the new process] are good. Using them is a different story . . .", while another said, "The [new process] is a good one. Some day I'd like to work on a project that actually uses it."

While this paper will largely draw on the experience of a single organization (described in Repenning 1996), the problem of process execution is widespread and increasingly well documented. For example, in the introduction of their second book, Wheelwright and Clark (1995) discuss the difficulty that many organizations experienced trying to implement the suggestions from their first volume (Wheelwright and Clark 1992). Similarly, recent studies of process improvement have produced a long list of initiatives that failed to achieve their stated objectives (Repenning 1996; Jones and Repenning 1997; Krahmer and Oliva 1996; Oliva *et al.* 1998). Unfortunately in many firms the new development process, which may have cost millions of dollars to develop, ends up being little more than a three-ring binder sitting on a shelf gathering dust.

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Reference modes and dynamic hypothesis

To understand the problem of process execution, the model development and analysis focus on two reference modes for the allocation of resources to development projects over time. The first mode, which the organization desired, is suggested by the development literature. A product development manager explains:

> We were trying to change [the traditional] mode by putting people up front with the [new] process, perhaps reaching a peak at 24 months or so before job one, and then reducing the allocation of resources until there are virtually none left on the project by job one.

Thus, the desired mode of execution is to allocate engineers to a project early in its development cycle and then reduce them as the project approaches completion. The rationale behind this profile is that there are a number of activities that, if done early in the development cycle, dramatically increase the effectiveness of design and development work. These activities, which include documenting customer requirements, establishing technological feasibility, and testing experimental prototypes, typically fall under the heading of *concept development* (Ulrich and Eppinger 1995).

In contrast to this desired mode, the same manager describes his organization's actual experience:

> [however] . . . if you look at our resource allocation on traditional projects, we always start late and don't put people on the projects soon enough . . . - then we load as many people on as it takes . . . the resource allocation peaks when we launch the project.

The problem with this mode of execution was that concept development activities were either done too late to be effective or were simply ignored, creating substantial rework late in the development cycle. Of course, resources are only a necessary condition for the successful completion of upstream activities. Time and experience are typically also required before organizations can execute these phases successfully. The dynamics associated with this feature will be examined later in the article.

Given the assumption that the source of difficulty lies, at least in part, in the allocation of resources to the up-front phases, the dynamic hypothesis focuses on explaining why this undesirable allocation of resources persists, despite the widespread recognition that it is sub-optimal. The same manager quoted above explains how the mismatch between the desired and actual resource profiles thwarted efforts to move the system towards the desired mode of execution:

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

> . . . there was a real dilemma because there were not people available to take off of all of these projects that have been developed in the traditional way where you were back-loaded with resources. By the very nature of the way we run the business . . . people were all out committed to launches.

Thus, organizations seeking to change their allocation of resources to favor the early portions of the development effort face a basic trade-off between completing the projects done under the old mode and allocating resources to newer projects.

The situation is, however, worse than this static description might suggest. Field studies suggest that any imbalance in resource allocation that favors fixing problems in downstream projects, rather than being corrected, is amplified until the organization invests little, if any, effort in upstream activities. One engineer in the Jones and Repenning (1997) study described the behavior created by this structure:

> . . . the completion date [of the design] is getting later and later each year. We are starving the ensuing model years to make the one we are on. We never have time to do a model year right, so we have lots of rework and so on.

The causal structure implicit in these explanations is captured in Figure 1. At the center of the diagram is a balancing loop that represents the resource allocation process. Managers are assumed to have a goal for the quality of each product introduced to the market. As a given development project approaches completion, managers assess the quality of that project via the number of defects remaining in the design. Upon perceiving a gap, managers respond by

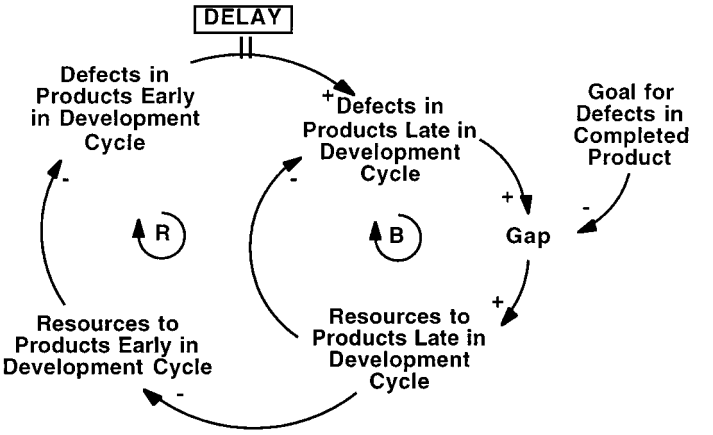


Fig. 1. Causal loop hypothesis

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

allocating more resources to the specific project. Additional resources lead to a reduction in the number of defects, thus closing the gap.

The assumption of resource scarcity is captured by the negative link from *Resources to Products Late in the Development Cycle* to *Resources to Products Early in the Development Cycle*. As the level of resources dedicated to a product early in its development process declines, the rate of defect reduction in that project also declines. These links create a positive feedback loop because the products later in the development cycle are eventually introduced to the market and those formerly in early portions of the development cycle move to the later stages of the process.

The existence and importance of this structure in multi-project development environments has been widely documented and discussed in the product development literature. For example, although they do not conceptualize it explicitly in terms of feedback loops, Wheelwright and Clark (1992) highlight this structure when, in discussing the potential problems associated with overloading a development system, they write:

> . . . if any one project runs into unexpected trouble, there is no slack available, and it will be necessary to take resources from other projects. This causes subsequent trouble on other projects and the effects cascade.

Similarly, Burchill and Fine (1997) find this dynamic at work in their studies of efforts to improve the concept development portion of the product development process, and Perlow (1999) identifies a similar dynamic in her study of time allocation and crisis management in a product engineering organization.

Despite the fact that the existence of such self-reinforcing dynamics has been widely acknowledged in product development, the phenomenon has not been the subject of a formal modeling effort. The lack of formal modeling leads to a number of limitations in the current understanding. First, while the existing literature typically asserts the existence of one or more such loops, it contains no discussion of the conditions required for the loop to dominate the behavior of the system in which it is embedded. The lack of boundary conditions is a severe limitation, as positive loops often exist but play little role in determining a system's dynamics. Understanding the conditions required for such a loop to dominate can also lead to additional insights. Second, few attempts have been made to describe the final outcome created by such self-reinforcing structures. Are they transient phenomena? Do they always cause the complete breakdown of a product development process or is there some intermediate final outcome? Qualitative treatments such as those of Perlow (1999) and Repenning and Sterman (2000) assert that such systems have steady-state operating modes, but do not discuss the negative loops that must ultimately constrain the otherwise

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

dominant positive loop. Third, how sensitive are these results to important parameters within the system? Qualitative models have little to offer on this question.

A model of multi-project product development

To answer these questions, in this section I present a model of resource allocation in a multi-project development environment. Few such models have appeared in the system dynamics literature (Weil *et al.* 1978 being the notable exception). Thus, although the model is based on a wealth of rich case data, I focus on the *simplest* possible model that can capture the dynamics of interest while still accurately capturing the relevant structure of real product development systems. Modelers with experience in this area will undoubtedly see numerous feedback processes that could be added. The collective understanding of these processes is sufficiently limited, however, that at this point additional detail would obscure the core process at work. Further, the simplicity of the formulation used allows me to use a variety of methods to understand the system's global behavior in some detail. Hopefully, as this line of inquiry progresses, a set of canonical formulations will emerge (as has happened with single-project models) and once these are identified and well understood, then the field will benefit from extending basic frameworks. I offer my formulations as only one of many candidate basic structures (another possibility is presented in Repenning 1999a) and challenge the field to both improve and extend the ideas developed here.

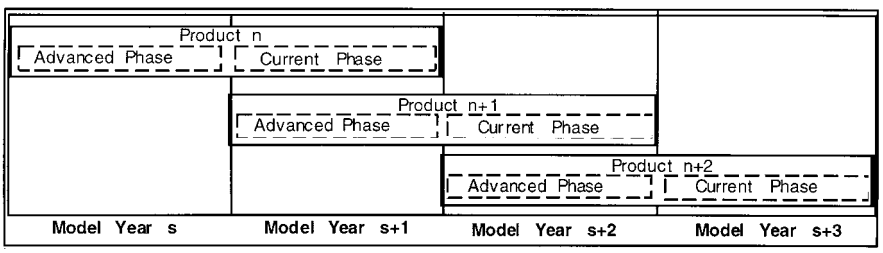
Model structure

OVERVIEW The model represents a simplified multi-project development environment in which a new product is introduced to the market every twelve months. The development cycle is assumed to be 24 months, so the organization always works on two projects at once. New projects are introduced into the system at 12 month intervals coinciding with the product launch date.

To develop a framework for understanding the dynamics surrounding multi-project management, I conceptualize the multi-project product development environment as a ''matrix'' organization in which there are two types of engineers: those assigned to projects and those assigned to functional departments (e.g. drafting, electrical, testing). I further assume that developing a product requires two distinctly different types of tasks. There are tasks directed at acquiring, translating, and coordinating information, and there are tasks

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Fig. 2. Project timing



directed at using that information to design the product's components. The first set of tasks is normally associated with the concept development and system-level design phases of a project, while the second set of tasks typically occurs in the detailed design and prototyping phases (see Cooper 1993 and Ulrich and Eppinger 1995 for detailed product development processes). I assume that project-specific engineers are responsible for the information acquisition, translation, and coordination tasks, while engineers within the functional areas are responsible for using that information for detailed design.

The distinction between the two types of engineers and tasks is important for two reasons. First, their dynamics are very different. Functional departments typically act as capacitated processing stations and are well captured by a queuing-based approach as in Adler *et al.* (1995). In contrast, engineers assigned to specific projects typically follow those projects through multiple phases. Further, in many development environments, engineers are assigned to multiple projects in different phases of the development cycle.

Second, engineers assigned to specific projects often have considerable latitude as to the point within the development cycle at which they generate the needed information. While the development literature has reached a clear consensus on the prescription that these tasks should be executed prior to detailed design (e.g., Cooper 1993), concept development and component design are often done more concurrently. To capture this latitude in task execution, I divide the 24 month development cycle into two phases. The first 12 month interval is labeled the *advanced* phase and the second 12 month interval is labeled the *current* phase. Tasks done in the advanced phase are assumed to be done *prior* to the execution of detailed design tasks, while tasks done in the current phase are assumed to be done concurrently with the relevant detailed design tasks. There is always one project in the advanced phase and one project in the current phase. The timing is shown in Figure 2.

KEY ASSUMPTIONS There are three key assumptions embedded in the structure of the model. First, the product launch date is fixed. This assumption is made

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<172::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

for computational and expository convenience, and, in many industries, it closely mirrors practice. US auto manufacturers typically introduce new vehicles on an annual cycle, as do manufacturers of recreational products such as motorcycles and snowmobiles. Further, competitive dynamics often dictate that products cannot be delayed for fear that valuable early sales (which may initiate many well known positive feedback loops) be ceded to a competitor. The value of this assumption is that it allows me to analyze the performance of the system via one output variable, the quality of the finished product. If the launch date is flexible, product quality must be traded off against time to market, thus requiring the specification of a more complex objective function to evaluate the system's performance.

Second, when resources are scarce (meaning that there are not enough project-specific engineers to execute all the tasks required by both projects), engineers give priority to the project in the downstream phase. The resource allocation rule represents an incentive scheme that was described by one project manager (in the Repenning and Sterman 2000 study) as ''Around here the only thing they shoot you for is missing product launch, everything else is negotiable.'' There are at least three reasons why organizations, sometimes despite their members' best intentions, might use such a policy. First, there is the firm's reputation and continued viability. Doing a concept development task, which prevents rework in future projects, does nothing to fix the problems in the project currently in the design phase. Second, in organizations focused on short-term profit and cash flow, projects closer to their introduction date represent a more immediate return on investment. Further, as projects reach their introduction dates, other investments, such as tooling and dedicated production lines, are often predicated on those dates.[1] Third, even apart from reputation and financial constraint, managers may still be biased towards downstream projects as a result of more basic features of human cognition, such as ambiguity aversion and the tendency to overweight salient and tangible features of the environment (see Repenning and Sterman 2000 for a detailed discussion). A manager in Repenning (1996) nicely described the problem created by differences in salience and ambiguity when he explained his organization's bias towards downstream work:

> . . . if you are engineer with a project close to job one, say 3 months, and for whatever reason you are not ready to ship your product, that's a very visible and apparent problem. There is no question the project is in trouble and needs attention. Now, if you are thirty months ahead of job one, 2.5 years away, and you are lacking customer definition. You are trying to get the same level of attention, ''I've got to have this or I can't move ahead'',

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

there much more of tendency to say ''Come on, quit crying and get on with it''.

The final assumption is that the probability of doing an information-related task incorrectly and creating rework is *higher* when that task is done in the current phase. The final assumption is critical to all the results developed and seemingly contrary to the ''classical'' project model, so it requires additional discussion. The features of product development that I am trying to capture are (1) the considerable latitude that project-specific engineering teams have in the timing of the information acquisition (e.g. documenting customer requirements) and translation (e.g. concept development) tasks with which they are charged, and (2) that the choice of such timing has important implications for the effectiveness of those activities and the amount of time that subsequently must be spent in information coordination activities. Specifically, the existing literature, and the recent popularity of the stage-gate development process (e.g., Cooper 1993), suggests that a development process works most effectively when information acquisition and translation tasks like concept development and system-level design are done prior to the design of individual components (Ulrich and Eppinger 1995). Further, the literature also suggests that teams that delay the acquisition of such information will consequently suffer higher error rates and attendant rework costs (Gupta and Wilemon 1990; Cooper 1993; Burchill and Fine 1997). The important implication of these findings for my model is that the high defect rates in detailed designs caused by incomplete and late information create extra work for both component designers (who are not explicitly modeled) *and* the project-specific engineers who must coordinate all the late changes and resolve conflicts among the various component designs.

While there is not sufficient space to give an equation by equation description of the model, the key structures and formulations are discussed below.[2]

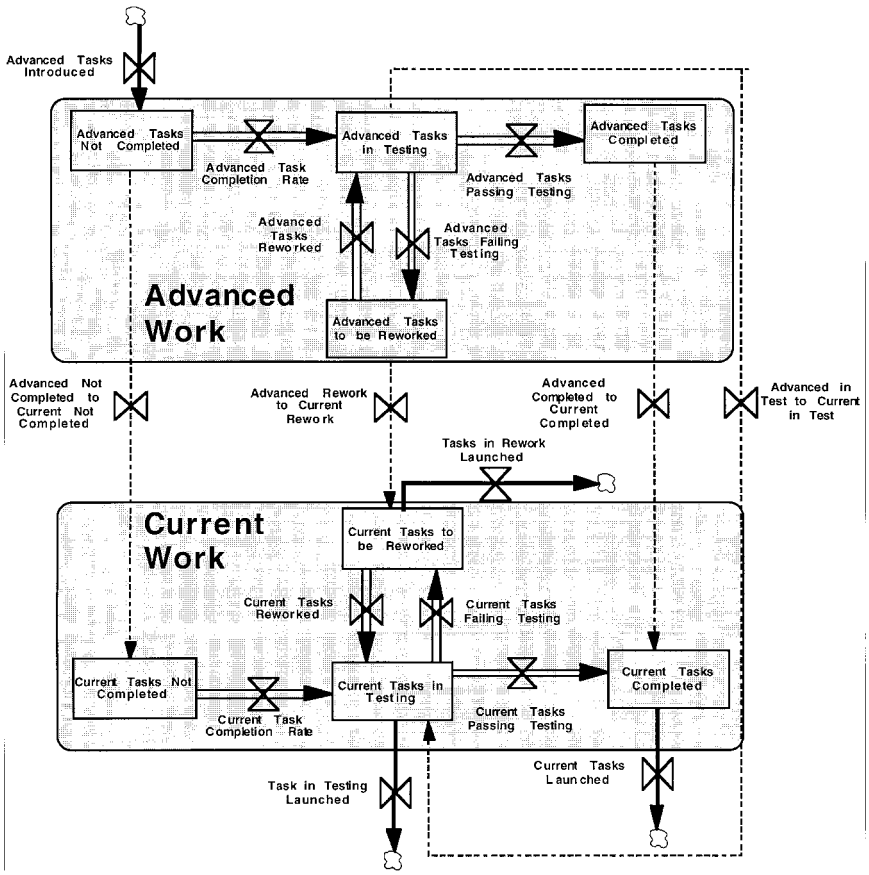
DETAILED STRUCTURE **Work accumulations and flows**

The basic stock and flow structure of the model is shown in Figure 3. Arrows with valve symbols represent the flow of tasks in the system. There are four stocks in each phase. The solid flow arrows represent the entry of new tasks into the advanced phase and the exit of the current project to the market. These flows are only active at the annual model year transition. The solid arrow at the top left of the figure represents the introduction of new tasks into the development system, which then accumulate in the stock of tasks in the advanced phase not completed.

The standard flow symbols (those that are hollow) represent the continuous

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Fig. 3. The basic stock and flow model



movement of work between the stocks during the given phase. The structure within each phase is identical. The stock of tasks not completed is drained by the task completion rate. Once completed, tasks accumulate in the stock of tasks in component design and testing. If a task passes the test, meaning it was done correctly and is not defective, it flows to the stock of tasks completed. Tasks failing testing flow to the stock of tasks requiring rework. Once reworked, tasks flow back to the stock of tasks awaiting testing.

Flows with double lines represent the continuous flow of work within a given model year. The dotted flow arrows represent the annual transition of work from the advanced to the current phase. Each stock of tasks in the advanced phase has such an outflow and each stock of tasks in the current phase has such an inflow. At the transition between model years, the entire contents of each work stock in the advanced phase is transferred to the corresponding stock of work in the current phase in one time step.

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Defects

The probability of doing a task incorrectly is determined by a slightly modified co-flow structure (Richardson and Pugh 1981; Sterman 2000). Each time a task is completed or re-worked there is some probability that it is done incorrectly and is defective. The number of defects introduced through new work and re-work is determined by multiplying the flow of tasks by the probability that a given task is done incorrectly. Once a task is completed or re-worked, its associated defect flows into a stock of defects in testing. Dividing this stock by the total number of tasks currently in testing determines the fraction of tasks that fail when tested. Tasks that fail testing remove one defect from the stock of defects in testing, while tasks that pass testing remove no defects from the stock. In real systems there are both type I and type II errors, but in the model testing is assumed to be perfectly accurate. Identical structures are used in both the advanced and current phases.

Allocation of capacity The critical decision function in the model is the allocation of engineering capacity between the four types of development work:

1. new tasks in the current phase (*CW*);
2. re-work in the current phase (*CR*);
3. new tasks in the advanced phase (*AW*);
4. re-work in the advanced phase (*AR*).

The allocation of effort between these categories is determined via a two-step process. First, a desired completion rate (DCR_i, $i \in \{CW, CR, AW, AR\}$) is determined for each type of work. Following the formulation often used in single-project models (e.g. Abdel-Hamid and Madnick 1991; Roberts 1964), the desired rate of work completion is calculated by dividing the stock of work to be completed by the number of months remaining until product launch. Thus:

$$DCR_i = \text{Work Remaining}/\text{\# of months to product launch.} \quad (1)$$

The formulation represents a decision process in which engineers look at the work to be completed and allocate it evenly over the time remaining.

In the simplest version of this formulation, engineers take no account of the chance that mistakes will be made and thus create additional rework. This assumption is somewhat unrealistic in product development since engineers often build prototypes in advance of the product's launch for the purpose of finding mistakes. To capture the idea that *some* rework may be anticipated, the four desired rates are formulated identically except for the rate of current work completion. In the formulation for the desired rate of completion of current

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

work the denominator, instead of being the number of months until launch, is the number of months until the prototype deadline (assumed to be six months prior to the product launch). Thus, in a simple way, engineers are assumed to factor in the possibility of making mistakes and plan their workload accordingly.

Second, once the desired completion rates are determined, the actual rates are calculated. Two rules are used:

1. new work takes priority over re-work;
2. work in the current phase takes priority over work in the advanced phase.

The actual rate of work completion in the current phase (*ACWR*) is the minimum of the maximum development capacity (*MDC*) and the desired completion rate for current work (DCR_{CW}):

$$ACWR = \mathrm{Min}(MDC, DCR_{CW}). \tag{2}$$

MDC is the maximum rate at which development work can be completed, and is equal to the total number of available engineer hours, *AEH*, divided by the minimum number of hours required per task (*MHT*):

$$MDC = AEH/MHT. \tag{3}$$

MDC does not measure the rate at which development work can be done properly, but rather the rate at which work can be completed when engineers are working as quickly as possible and skipping all steps that do not directly contribute to finishing the design. Such skipped steps might include documenting work and testing new designs.

If, after the current new work has been completed, there is capacity remaining, then it is allocated to current rework. The actual rate of current re-work (*ACRR*) is equal to minimum of capacity remaining, *MDC-ACWR*, and the desired rate of current re-work (DCR_{CR}):

$$ACRR = \mathrm{Min}(MDC - ACWR, DCR_{CR}). \tag{4}$$

Capacity is allocated to the advanced phase in a similar fashion. First, the capacity that *can* be allocated to the advanced phase (*CAT*) is determined by subtracting the capacity allocated to current tasks from the normal development capacity (*NDC*). *NDC* is the rate at which engineers can complete tasks when they are following the prescribed development process. Thus:

$$CAT = \mathrm{Max}(NDC - (ACWR + ACCR), 0). \tag{5}$$

The maximum function is important because the rate of work completion in the current phase can exceed the normal development capacity (since

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

MDC > *NDC*). If capacity does remain, then it is allocated to advanced work and advanced re-work. Normal development capacity is equal to the available engineer hours (*AEH*) divided by the normal hours required per engineering tasks (*NHT*):

$$NDC = AEH/NHT. \tag{6}$$

This formulation embodies an important assumption: advanced work does not get completed unless the current work rate is less than the normal development capacity. If the normal development capacity is insufficient to complete all the outstanding current tasks, then engineers will reduce their time per task to reach the required work rate. This does not happen, however, for advanced work. If the organization is working beyond its normal capacity, advanced work is not done.

Model behavior

BASE CASE Figures 4 and 5 show the base case behavior[3] for task completion in the advanced and current phases.

At each 12 month interval 1500 new tasks, constituting a new development project, are introduced. Work in the advanced phase is completed at a constant

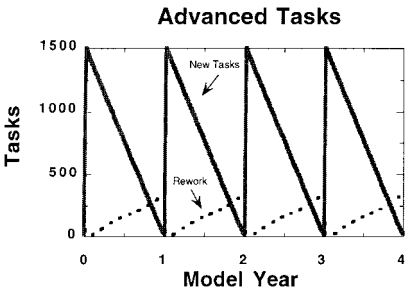


Fig. 4. Base case behavior in the advanced phase

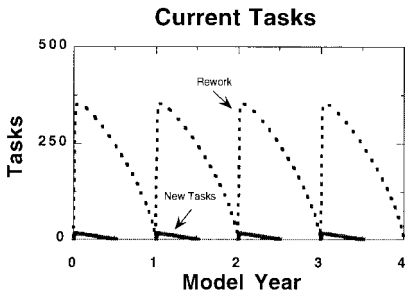


Fig. 5. Base case behavior in the current phase

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

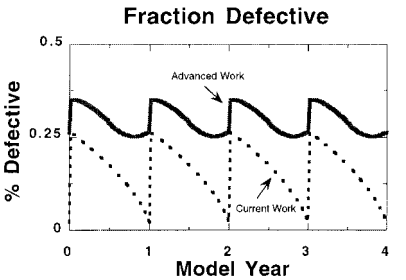


Fig. 6. Average defect level in advanced and current phases

rate throughout the year, so the stock of advanced tasks to be completed declines approximately linearly. Most new tasks are completed in the advanced phase and only a small number (less than 50) are sent to the current phase uncompleted. The level of re-work to be completed grows during the advanced phase as tasks done incorrectly are discovered through testing.

Once the project reaches the current phase, the small number of tasks received uncompleted are quickly finished. Approximately 300 tasks requiring rework are also received from the advanced phase. The rate of re-work completion rises through the year as the product launch date approaches and the organization increasingly focuses on current work in the hope of fixing all the defective tasks before the product's launch.

Figure 6 shows the average defect level in the advanced and current phases. The defect fraction in advanced work starts at approximately 35% (the probability of doing an advanced task incorrectly) and declines for the first nine months of the year. It then increases slightly as the rate of re-work completion in the advanced phase drops because of lack of resources. The fraction of defective tasks in the current phase drops dramatically throughout the year as a result of multiple iterations through the rework cycle.

Figure 7 shows the allocation of resources between the two phases. At the beginning of the year approximately 25% of the total capacity is allocated to the project in the current phase. The allocation remains approximately constant

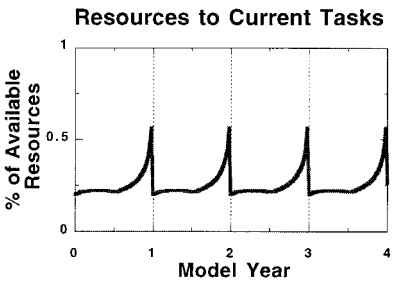


Fig. 7. Allocation of resources between the two phases

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

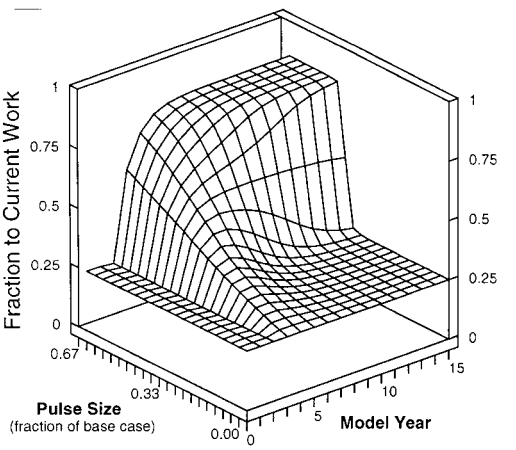


Fig. 8. The effect of different pulse sizes

for the first half of the year and then increases as the product launch date approaches. The shift in the allocation of resources leads to an increasing rate of defect reduction in the current phase (Figure 6).

CAPTURING THE REFERENCES MODES The base case depicts an organization that operates in a desirable mode: resources are allocated to projects early in their development cycles, and most information-related work is done prior to the detailed design phase. Thus, while the model displays the desirable mode of execution, it has yet to show the actual mode experienced by many organizations. An instructive demonstration of the range of behaviors produced by this system is provided by introducing a one-time increase in the amount of development work. Figure 8 shows the effect of different pulse sizes (introduced in model year one) on the average fraction of the resource base that is allocated to the current project. Figure 9 shows the impact of the additional workload on system performance.

The pulse tests show two important features of the system's behavior. First, the model is capable of generating both modes of process execution. For pulses greater than one-third of the base case, the system does not return to initial, desirable equilibrium. Instead, the fraction of resources dedicated to the current program grows dramatically, and the system settles into a new steady-state in which little effort is dedicated to upstream activities (Figure 8). As a consequence, system performance declines dramatically (Figure 9).

Second, the test also shows that a transient shock, if it is large enough, is sufficient to cause a permanent change in steady-state behavior. Thus, the

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

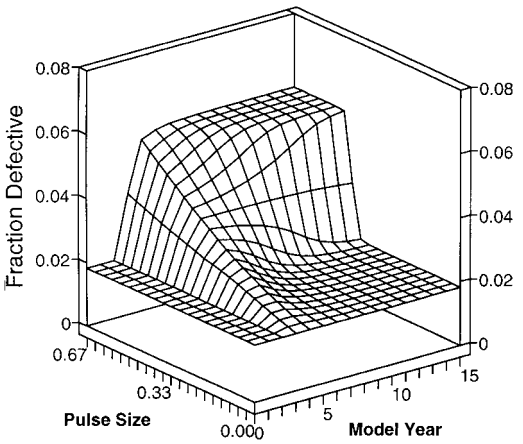


Fig. 9. The impact of the additional workload on system performance

model provides one theory for why development organizations may get stuck in undesirable modes of execution.

Figures 10 through 13, which show a selected simulation run (a pulse equal to 50% of the base case value) help identify the source of the undesirable behavior mode.

In the year of the pulse there are 2250 tasks to be completed in the advanced

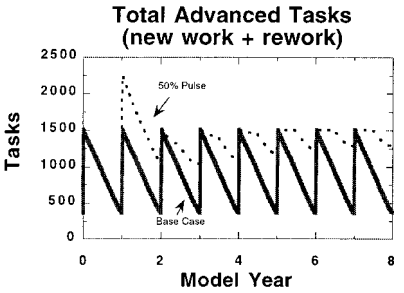


Fig. 10. Simulation: Total advanced tasks

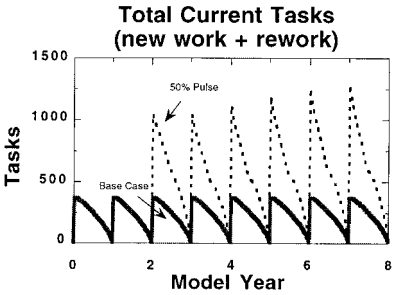


Fig. 11. Simulation: Total current tasks

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

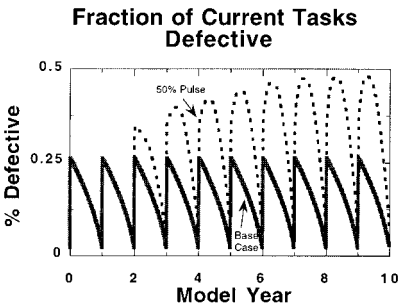


Fig. 12. Simulation: Fraction of current tasks defective

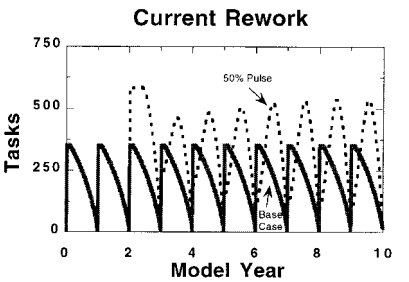


Fig. 13. Simulation: current rework

phase rather than 1500. During that year, the number of tasks completed in both the advanced and current phases does not change. Since there has been no change in the completion rates at the end of the model year, due to the increased workload, 500 additional tasks move to the current stage uncompleted (Figure 11). In model year two, due to the 500 additional tasks in the current phase, the completion rate in the advanced phase declines (Figure 10). Since the probability of introducing a defect is higher in the current phase, the additional uncompleted tasks lead to a higher defect fraction in the current period (Figure 12), thus creating more rework (Figure 13). The extra rework requires additional resources that would not have been needed *had those tasks been done in the advanced phase*. The additional resource requirement in the current phase further reduces the task completion rate in the advanced phase, which, in turn, causes the situation to be even worse in subsequent model years. Thus, the undesirable behavior mode is created by a positive feedback loop that drives the organization to allocate an increasing fraction of its resources to fix problems in the current phase and away from the activities early in the development cycle that prevent those problems.

BOUNDARY CONDITIONS AND SENSITIVITY ANALYSIS While the dynamic hypothesis is capable of generating the mode of interest, much about this phenomenon

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

remains to be understood. In particular, for this theory to be useful, the boundary conditions on the phenomenon of interest need to be established and the sensitivity to key parameters assessed. I use a two-step strategy to analyze the global behavior of the model. First, I approximate the behavior of the full system with a simpler one-dimensional model that can be more easily analyzed. Second, to establish the validity of these approximations, I conduct extensive sensitivity analysis (a subset of which is presented here). The role of the simpler model is twofold. First, it provides insight into the behavior of the full system; second, it guides the search for interesting dynamics in the full model.

Reduced form analysis

To simplify the model I eliminate the testing delays and assume that engineers plan their workload accordingly. With these assumptions it is possible to reduce the system to a one-dimensional map (see Appendix for details). Let s index the model year, let P_A be the probability of creating rework in the advanced phase, let P_C be the probability of creating rework in the current phase, let A represent the total number of tasks in a project, and let $f(s)$ represent the fraction of the work that is accomplished in the advanced phase in model year s. Then, with suitable approximations to the within-year dynamics, the system can be reduced to the following map in $f(s)$:

$$f(s) = \mathrm{Min}\left(1, \mathrm{Max}\left(\left(\frac{NDC \cdot T}{A/(1-P_A)} - \frac{1-P_A}{1-P_C}\right) + \frac{1-P_A}{1-P_C} \cdot f(s-1), 0\right)\right).$$

This map captures the dynamics of the simplified system by relating the fraction of advanced work that is completed in a given model year to the fraction that will be completed in the next model year.

Analysis of this equation suggests at least three insights into the behavior of the larger system. First, some algebra yields the conditions required for each of the two equilibria to exist. An equilibrium exists at the desirable operating mode, $f(1) = 1$, if and only if:

$$NDC \cdot T \geqslant \frac{A}{1-P_A}.$$

The right-hand side of this equation represents the number of tasks required to develop a defect-free product when all tasks are done in the advanced phase. Thus, a necessary condition for the organization to operate in the desired mode is that it has sufficient resources to do so.

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Similarly, an equilibrium exists at the undesirable mode of execution, $f(0) = 0$, if and only if:

$$NDC \cdot T \leqslant \frac{A}{1 - P_C}.$$

The right-hand side of this equation represents the number of tasks required to develop a defect-free product when all tasks are done in the current phase. Thus, a necessary condition for the organization to operate in the undesirable mode (in steady state) is that resources are insufficient to complete all the tasks when doing so. Combining the two equations shows that for both equilibria to exist simultaneously, the following must also hold:

$$\frac{1 - P_A}{1 - P_C} \geqslant 1.$$

Inspection of the equation for $f(s)$ shows that the term on the left-hand side of the above inequality is equal to the slope of the line relating $f(s)$ and $f(s-1)$. If the slope of this line is greater than one, then the positive feedback loop dominates the behavior of the system. Some further algebra shows that this condition reduces to $P_C > P_A$. Thus, in the simplified model, the conditions required for both modes of interest to exist are:

1. the probability of introducing a defect must be lower in the upstream phase;
2. capacity is sufficient when all tasks are executed in the upstream phase;
3. capacity is insufficient when all tasks are postponed to the downstream phase.

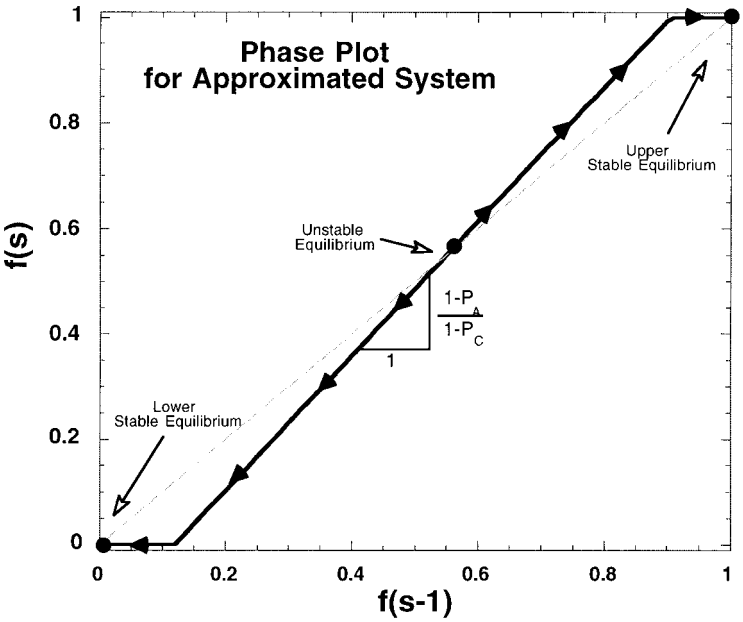
A sample phase plot, in which the required conditions are satisfied, is shown in Figure 14.

Second, analysis of the simplified system also suggests the outcomes that are obtained when one or more of these conditions are not satisfied. If $P_C \leqslant P_A$, then the slope of the phase plot is less than one and the system has a unique, stable, interior equilibrium. If capacity is insufficient regardless of the mode of execution, then the system again has a unique stable equilibrium at $f(0) = 0$, regardless of the relationship between the two defect probabilities. Similarly, if capacity is sufficient in either case, then the system again has a unique equilibrium at $f(1) = 1$.

Third, and most importantly, the simplified model also yields some insight into the system's disequilibrium dynamics. As the phase plot indicates, when the positive loop dominates and the conditions required for the existence of both equilibria are satisfied, then the system has a third, unstable equilibrium (located at the intersection of line relating $f(s)$ to $f(s-1)$ and the 45 degree

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Fig. 14. A sample phase plot



line). While the system will never settle at this point, it does represent the point at which the dominant positive loop changes direction and the system begins to move towards the other equilibrium. Thus, the location of the unstable equilibrium determines the size of the shock required to move the system from one equilibrium into a new trajectory that ends at the other equilibrium. Solving the map for $f(s)$ yields the following expression for the location of the unstable equilibrium:

$$f^*(s) = \left(\frac{1-P_A}{P_C-P_A}\right)\cdot\frac{\left(\frac{A}{1-P_C} - NDC\cdot T\right)}{\frac{A}{1-P_C}}.$$

As the equation shows, the location of the unstable equilibrium is determined by the utilization of capacity (as represented by the second term on the right-hand side). As the amount of work in the system increases relative to the available resources, the unstable equilibrium moves closer to $f(s) = 1$. Eventually, capacity becomes insufficient regardless of the mode of execution and the system has a unique equilibrium at $f(0) = 0$. Similarly, as the utilization decreases, the unstable equilibrium moves towards $f(0) = 0$, and, when capacity is sufficient regardless of the mode of execution, then the system has a unique

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

equilibrium at $f(1) = 1$. Thus, the analysis suggests that the robustness of the system to the downward spiral created by external shocks is determined by the utilization of resources in the steady state: as utilization increases, a progressively smaller shock is needed to push the system over the unstable equilibrium, causing the dominant positive loop to work in the undesirable direction, and thus trapping the system in a state of low performance.

Robustness of reduced-form results

The results so far have been developed within a reduced model that contains no testing delays. In this section, simulations of the full model are presented to test the validity of these approximations and to determine if the insights from the reduced-form model carry over to the larger system. Figure 15 shows the actual phase trajectories for a variety of pulse inputs. Like the reduced-form model, the full system has two stable equilibria (highlighted by the two small back dots). In contrast to the reduced system, the stable equilibria are not at the extreme points but, for this particular set of parameters, exist at approximately $f(s) = 0.2$ and $f(s) = 0.7$. The system also has an unstable equi-

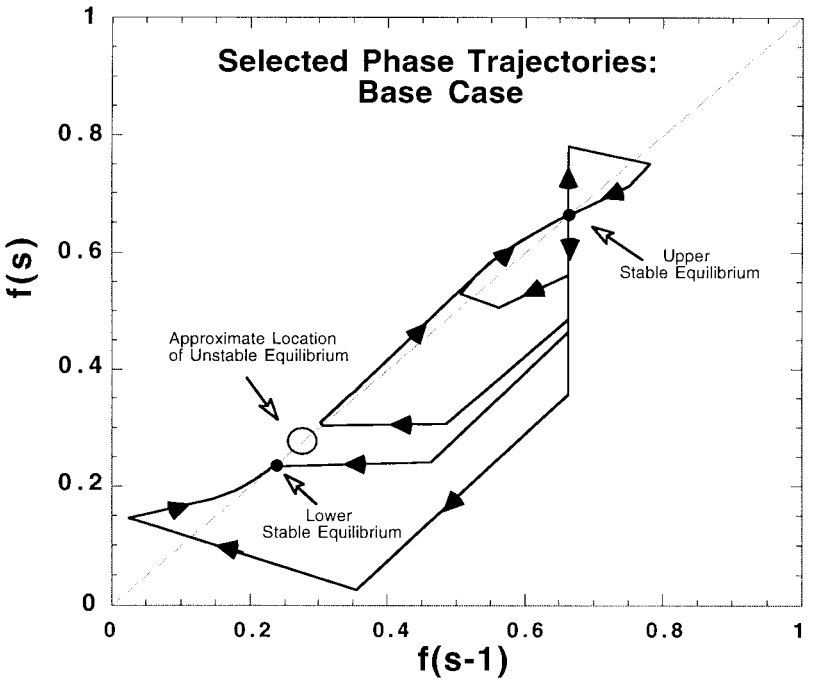


Fig. 15. Actual phase trajectories for different pulse inputs

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

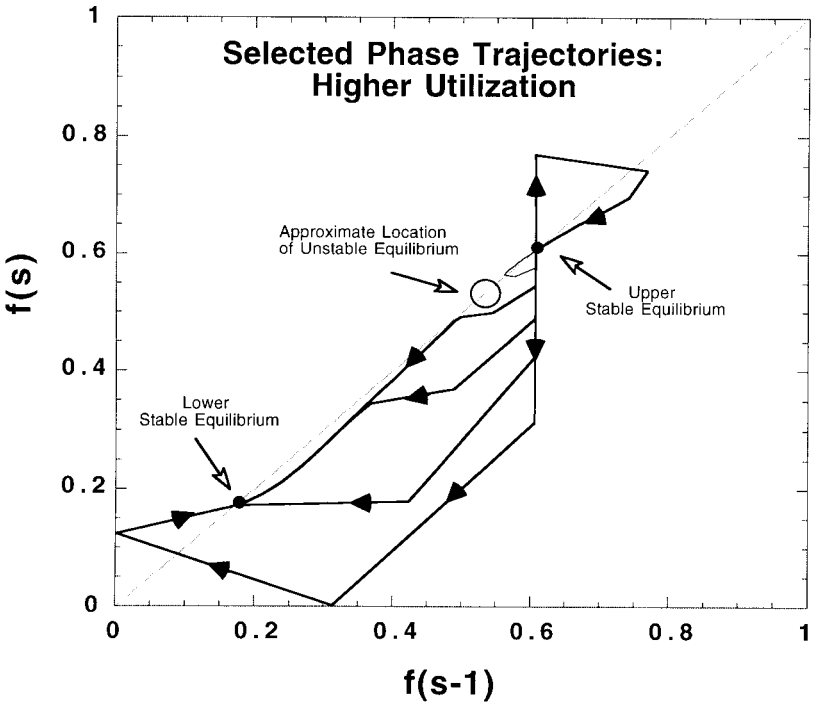


Fig. 16. The result with increased resource ultilization

librium (its approximate location is highlighted by the open circle). The unstable equilibrium of the full system is much closer to the lower stable equilibrium, suggesting that the existence of the within-year delays adds stability to the system. The global dynamics are, however, similar to those of the simplified model.

To confirm the insights generated in the simple model, Figures 16 and 17 show the results of experiments when the resource utilization is increased and decreased. Here the results of the simple model apply directly to the larger system. Figure 16 shows that, as utilization increases, the unstable equilibrium moves towards the desired stable equilibrium. Additional simulations (not presented here) show that increasing utilization further causes the desired equilibrium to disappear altogether.

Similarly, as resource utilization is decreased, the unstable equilibrium moves closer to the lower stable equilibrium. As demonstrated in Figure 17, if utilization is decreased enough, both the unstable equilibrium and the lower stable equilibrium disappear. In this case, all trajectories return to the initial equilibrium regardless of the size of the initial shock.

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

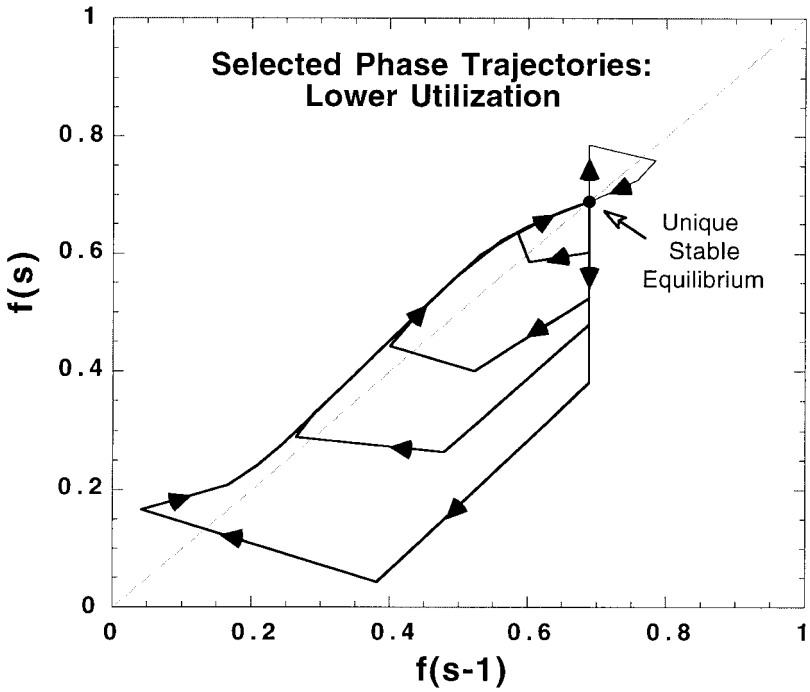


Fig. 17. The result with decreased resource ultilization

The role of within-year testing delays

One feature of the full system not captured in the reduced-form analysis is the within-year testing delays. To analyze the role of these delays in determining the system's behavior, Figure 18 shows selected phase trajectories for the system when the testing delays are decreased by a factor of four; Figure 19 shows trajectories for the system when the testing delays are increased by a factor of four.

Figure 18 shows that as the delays are reduced, the system's phase map begins to match the simpler model more closely. As the delays are reduced, the unstable equilibrium moves to the right and a smaller shock is needed to push the system into a new trajectory. In contrast, as the testing delays are lengthened, the unstable equilibrium moves to the left and the system becomes less prone to the self-reinforcing dynamics. Figure 19 shows that if the testing delays are sufficiently long, then the unstable equilibrium disappears and the system has a unique steady-state operating mode. Thus, as conjectured above, the within-year testing delays add stability to the system.

The intuition behind this result is best explained with an extreme-condition

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

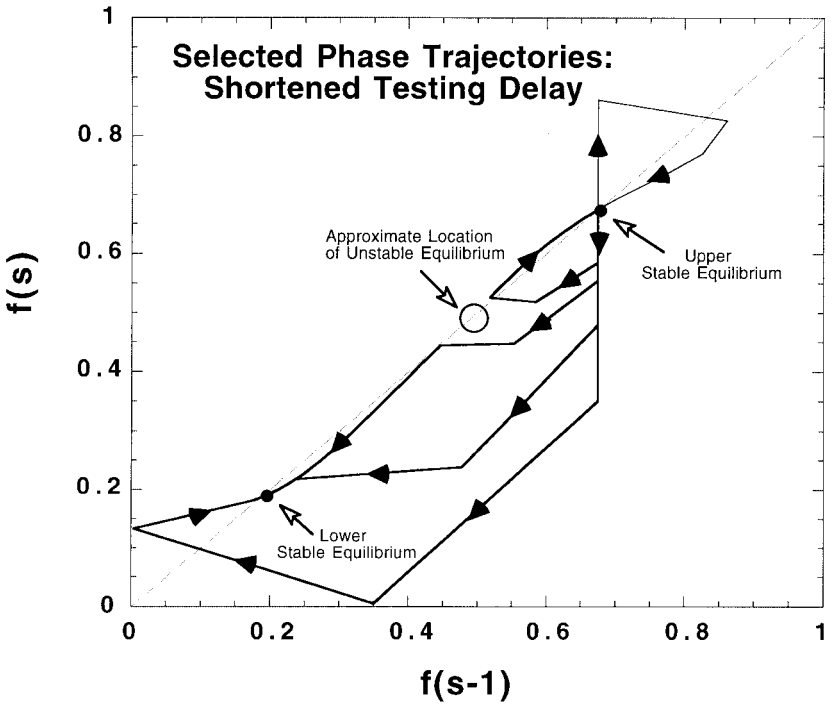


Fig. 18. Phase trajectories when testing delays are decreased by a factor of four

thought experiment: what happens if the testing delays are infinitely long? In this case, each task only gets done once. While the system performance would be better if all tasks were executed in the advanced phase, under this scenario the resource requirements are exactly the same in either phase and, thus, the positive loop no longer exists. Further, if the testing delays are infinite, capacity is effectively increased since there is no rework. Similarly, as the testing delays get longer, the positive loop is weakened to the point that it no longer dominates the system. It is important to note, however, that the analysis does not suggest that lengthening these delays is a wise policy. In fact, lengthening the testing delays dramatically reduces the system's overall performance since it slows the rework process.

IMPLICATIONS FOR PRACTICE The results of the analysis can be summarized as follows: Product development systems can have multiple steady-state modes of process execution, and such systems can get trapped in a state of low performance from which, without any outside intervention, they will not recover. The critical condition that drives this result is the existence of tasks that are more effectively done in the early stages of a development process ($P_A < P_C$).

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

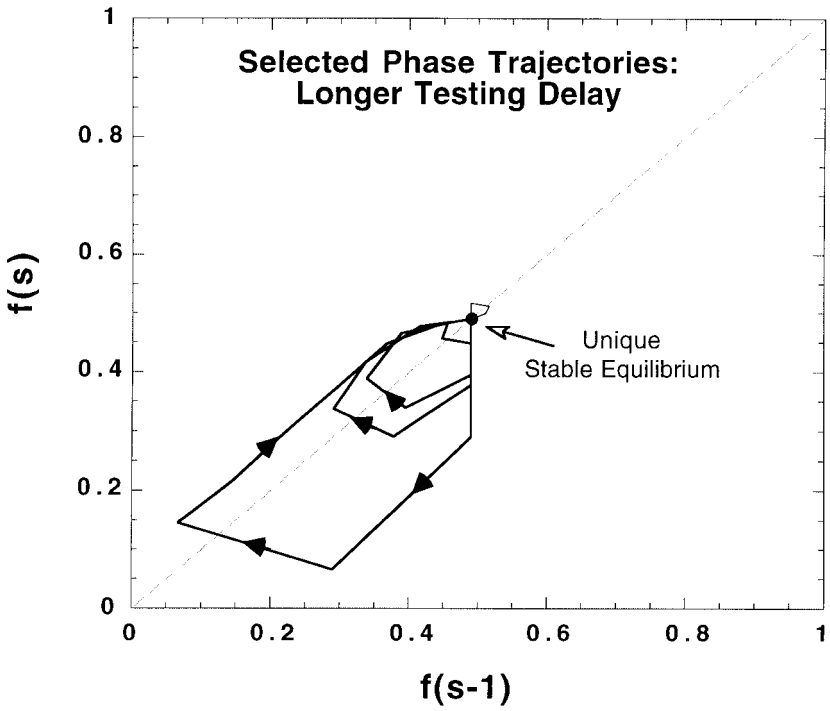


Fig. 19. Phase trajectories when testing delays are increased by a factor of four

The analysis also demonstrates that resource utilization determines how prone the system is to undesirable self-reinforcing dynamics, and suggests that delays in discovering rework add stability to the system.

These insights have at least three implications for practice. First, they suggest that capacity planning is critically important in a product development system. Although the test just shown was a pulse, it should be fairly obvious that a permanent increase in workload (a step) of even a modest size is sufficient to initiate a downward spiral and dramatically degrade the system's performance. This cost of overloading has already been well recognized in the literature (e.g., Wheelwright and Clark 1992). More subtly, however, the analysis also demonstrates the need for a *dynamic* approach to resource planning. At the outset of a given project, resources may be well matched to the project's known requirements. Projects, however, often require more resources than were initially anticipated as a result of changes in scope or major design problems. Such contingencies constitute a transient increase in workload, and, if resource plans are not rapidly updated to account for them, they may be sufficient to initiate the downward spiral and cause a permanent decline in system performance. Wheelwright and Clark (1992) suggest that organizations keep

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

reserve capacity to accommodate such changes. An alternative, and complementary, approach is to regularly update the resource plans for all projects as new information about the requirements of projects in their downstream phases is obtained. Somewhat surprisingly, few methods for such a dynamic approach to resource planning are available in the existing literature. More detailed, organization-specific, SD models may have much to contribute in developing such a methodology.

Second, the analysis suggests that the system can operate in an undesirable mode, even when the resources and the workload are well matched. This possibility has not been discussed in the existing literature. The pulse test shows that a single shock can move the system to a state from which it will not return. Thus, managers need to make resource allocation decisions based on the *current* mode of process execution, not the mode that they desire. Failure to acknowledge that a process is not operating as desired can lead to the under-allocation of resources, thereby trapping the system in such an undesirable state.

Third, managers have become increasingly focused on cycle-time reduction. The analysis suggests that care is required when reducing the delays inherent in rework discovery. Reducing such delays unequivocally improves system performance given a particular mode of execution. Reducing testing delays, however, also makes the system more prone to changing modes. Thus, while managers can benefit from increasing the speed of the rework loop, they must be aware that they are creating a system that is more prone to undesirable self-reinforcing dynamics and thus must make their resource plans more carefully.

Extensions: introducing process improvement

The model as formulated provides one explanation for why organizations often operate in a high-stress, high-rework mode and experience difficulties in following a prescribed development process. In this section I present one extension to the basic model. Specifically, a common response to low performance is introducing modifications to the engineering and design process. Such changes include:

- new tools to reduce the probability of introducing a defect and to speed the rework cycle;
- specification of a modified sequence of development steps including appropriate checkpoints and reviews;
- the introduction of more stringent standards for documentation.

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Unfortunately, the existing empirical literature suggests that specifying a better process is not sufficient to insure that it is used effectively. The failure of such *process improvement* initiatives has recently become of a topic of interest in the SD literature, which now contains a number of models that highlight the difficulties associated with trying to improve and change one or more processes within an organization (e.g., Krahmer and Oliva 1996; Sterman *et al*. 1997; Oliva *et al*. 1998; Repenning 1999b; Repenning and Sterman 2000). A common theme highlighted in all of these studies is the failure of managers to understand the dynamic implications of process change, and, as a consequence, the failure of initiatives due to the unintended side effects of well-intentioned actions. Such a finding should be of little surprise to SD modelers since process improvement and change are inherently disequilibrium phenomena and the limited ability of humans to manage such dynamics lies at the foundation of the field (Forrester 1961).

A limitation of each of these studies in the present context is that they focus either specifically on manufacturing environments or on improvement efforts in a generic context. In each case, the model does not contain detail specific to the product development process (an important exception is that all recognize that improvement in product development may take longer). Thus, while one should expect that managers in product development settings are likely to fall prey to many of the failure modes identified in the other studies of process improvement, the domain-specific nature of these failure modes remains to be identified and analyzed. To that end, in this section, additional structure is added to the model to capture the introduction of new tools and processes.

Introducing a new tool or process is assumed to have two impacts on the system. First, the new technology reduces the probability of introducing a defect in either phase. The full benefit of the tool, however, is not realized until the organization has developed experience using it; there is a delay between beginning to use a new tool and receiving its full benefit. Second, using the new technology may make completing each development task take *longer*. The increase in task duration captures the fact that new tools and processes have additional documentation and reporting requirements. The model is parameterized in such a manner that the reduction in the probability of introducing a defect more than compensates for the increase in task duration. Thus, if the firm can gain the necessary experience, adopting the new methodology increases the total capability of the development system. The basic dynamic of introducing such tools was aptly described by one manager, who explained the trade-offs associated with following a new process that required better documentation of computer code:

[The process] clearly takes more time and more resources because writing

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

> the code on the back of an envelope is a lot faster than documenting it. Of course the quality of code went up if you documented it and fixed things that might require rework later, but that only shows up in speed after the fact. It takes you longer initially but overall it takes less time.

In this section, the basic model is extended to study how the introduction of such tools changes the system's behavior.

MODEL STRUCTURE The introduction of a process improvement initiative is captured in a simple way. First, there is assumed to be a threshold allocation of resources to the advanced phase required to achieve the full benefit of the new tools. The assumption is based on the fact that many of the current innovations in product development are focused on the up-front portion of the development process and have little impact if they are used later in the development cycle. For example, computer simulation methods for assessing product designs are of less incremental benefit when used concurrently with physical prototypes. Conversely, they can be very beneficial if used in the early stages of the process before such prototypes are needed. For the purpose of this model I assume that, to achieve the full benefit of the new tools, the organization must allocate 75% of its total resources to the advanced phase (the initial value in the base case). The organization's current adherence to the methodology (*CAM*) is then calculated by dividing the current fraction of resources (*FRA*) dedicated to the advanced phase by the threshold value (*FRA**). Thus:

$$CAM = \text{Min}\,(1, FRA/FRA^*). \tag{7}$$

The rate of defect introduction is then determined by the organization's average adherence to the methodology (*AAM*). *AAM* is an average of the current adherence weighted by the number of changes that have been made following the methodology in the past three model years. A co-flow structure is used to calculate this weighted average (see model documentation for details).

The impact of the new methodology on capacity is captured by changing the equation for normal development capacity (*NDC*). Once the new technology is introduced, *NDC* is given by:

$$NDC = AEH/DHT. \tag{8}$$

DHT represents the time required to complete a task while following the methodology.

MODEL BEHAVIOR The new development tools and processes are introduced in

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

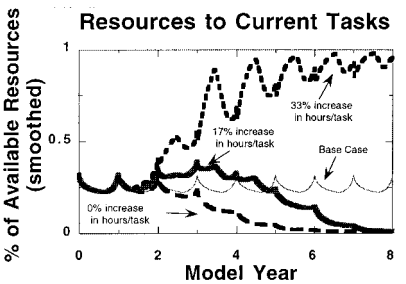


Fig. 20. The effect on resources of introducing a new methodology

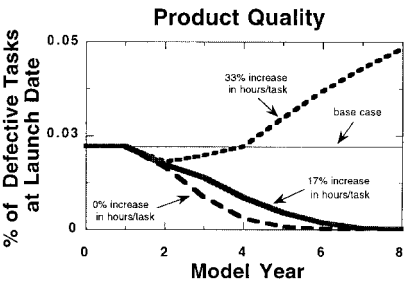


Fig. 21. The effect on product quality of introducing a new methodology

the first model year. Figures 20 and 21 show the consequence of introducing a new methodology under three different assumptions about the necessary change in normal development capacity. In the first, the methodology requires no more time than the current operating procedure and thus normal capacity is not reduced: $NHT = DHT$. In the second, the methodology increases the required time per task by one sixth; in the third, the time required per task is increased by one third.

If the methodology entails no sacrifice in capacity (case 1), then the fraction of work devoted to current programs decreases immediately after the methodology is introduced. As more work is shifted to the advanced phase, fewer defects are introduced and less re-work is required, enabling an even larger fraction of work to be completed in the advanced phase. If the cycle works in this direction, the system's performance improves monotonically and the dominant positive loop is instrumental in shifting resources towards the upfront portion of the development process. In case 2, the decrease in development capacity causes the fraction of resources dedicated to the current phase to increase initially. The change is, however, only temporary. After three model years, the fraction begins to decline and the new methodology again leads to a substantial reduction in the number of defects introduced.

Case 3 shows different behavior. Whereas in cases 1 and 2 the introduction of the new methodology leads to an improvement in system performance, in

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

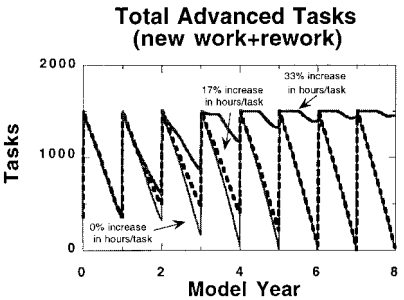


Fig. 22. Total advanced tasks following the introduction of a new methodology

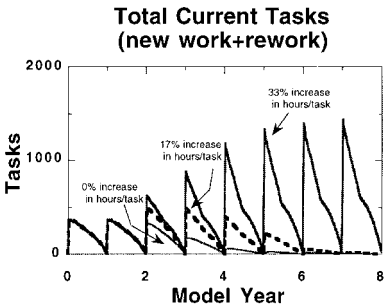


Fig. 23. Total current tasks following the introduction of a new methodology

case 3 it leads to a substantial decline, even though the model is parameterized so that the new methodology, if fully adopted, leads to an improvement in capability. Figures 20 and 21 show that in case 3, the system evolves to the point where the vast majority of the work is done in the current phase and the defect fraction increases substantially.

Figures 22 and 23 help explain the different outcomes. In both cases 2 and 3, because of the increase in the time required per task, fewer tasks are completed in the advanced phase in the years following the introduction of the new methodology (Figure 22). As a consequence, more uncompleted work is moved to the current phase (Figure 23). As in earlier cases, more work in the current phase leads to a higher defect introduction rate (Figure 24). In case 2, this change is temporary. As the benefits of the new methodology kick in (Figure 25), the defect introduction rate falls, fewer resources are needed for the current phase, and the advanced work completion rate begins to rise. As the advanced work rate rises, adherence to the methodology increases and performance improves as the positive feedback loop works in the desired direction.

In contrast, in case 3 the advanced work completion rate does not recover. Instead, as more work is shifted to the current phase (Figure 23), the defect

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

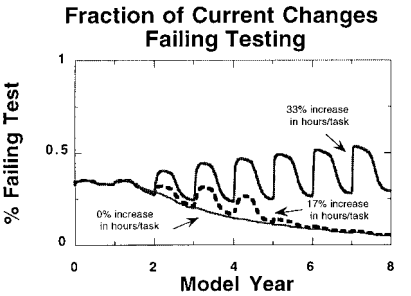


Fig. 24. Fraction of current changes failing testing following the introduction of a new methodology

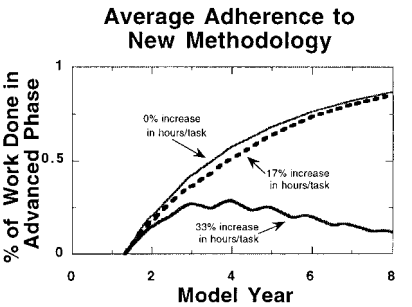


Fig. 25. Average adherence to a new methodology

introduction fraction rises enough to offset the benefits of the new tools and processes, creating even more work in the current phase (Figure 24). In this case, the initial transient is large enough that the system never recovers. The decline in performance is reinforced as an increasing fraction of resources are allocated to current programs and adherence to the methodology declines (Figure 25). The initial shock created by the longer time per task is large enough to push the system over the unstable equilibrium and, as a consequence, the positive loop works against the implementation of the new methodology by reducing adherence to the new process and pushing resources to the downstream portion of the development cycle.

IMPLICATIONS FOR PRACTICE These simulation experiments make an important point. The naive introduction of a new set of tools in product development that has the *potential* to improve performance can actually make performance worse rather than better. This dynamic played a significant role in creating the difficulties experienced by the company discussed in the introduction. In an effort to improve its product development process, the organization spent millions of dollars introducing a new computer-aided design system, but did not provide the necessary release time and resources to allow engineers to actually develop the appropriate skills with the tools. Initially, many engineers tried to

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<172::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

use the new systems, but unfamiliarity with the new tools slowed their progress and caused them to fall behind in their normal development work. One engineer explained his experience:

> ...I had some background in CAD/CAE from my master's program and I still stayed at work until midnight every night for a month learning how to use the tools and trying to figure out how to get my work done... some of the older engineers, even with training, they just have a [computer] sitting on their desks gathering dust... there are still three guys in my section that don't use the tools.

Most eventually abandoned the tools in favor of the projects for which they were responsible, and many reported that the performance of their projects suffered for having tried to incorporate the new process technology. This story is unfortunately all too common, but rarely do those who promote new tools and methods worry that their introduction may make the system worse rather than better. As the analysis shows, however, in an environment with scarce resources this is a distinct possibility. Managers implementing new tools and processes need to recognize this ''worse-before-better'' behavior mode and make resource plans accordingly.

Conclusion and future directions

Many authors have recognized that managers who treat development projects as independent entities do so at their peril (e.g., Cusumano and Nobeoka 1998; Sanderson and Uzimeri 1997). One of the major thrusts of Wheelwright and Clark (1992, 1995) is that senior leaders should focus on managing the portfolio of development efforts rather than individual projects. It has become an article of faith among business writers that managers should focus on whole systems and processes, not functions and specific pieces (Senge 1990; Stata 1989; de Geuss 1988). This shift in perspective is obviously important, but the literature contains less guidance on how to put such thoughts into action. Unfortunately, more is needed than simply recognizing that product development processes are complex systems with interdependent elements. A substantial body of research shows that human decision-makers perform poorly in dynamic environments (Paich and Sterman 1993; Diehl and Sterman 1995; Sterman 1989a, 1989b; Brehmer 1992; Funke 1991).

The analysis presented here focuses on one decision, made in the context of product development, that requires such a systemic perspective: the allocation of resources between current and future projects. There is a growing body of

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

both data and theory that suggests that people behave myopically in these contexts, specifically over-investing in current projects and ignoring longer term investments (Repenning and Sterman 2000; Jones 1997). Combining resource scarcity and myopic behavior creates a system with at least two undesirable characteristics: the system is not robust to variations in workload and new tools and processes are difficult to implement. These results suggest that in a multi-project development environment more is needed than simply acknowledging that product development is a system.

Instead, product development managers need a set of models, methods, and tools that SD is uniquely positioned to provide. Future work in SD could provide substantial benefit in at least three areas. First, the dynamics of multi-project development environments are far from well understood. The model developed here is highly stylized and contains a number of simplifying assumptions. Foremost among these are the fixed launch date and representing testing as a delay. Many firms have considerable latitude in the dates at which they launch new products, and this additional degree of freedom may change the dynamics discussed here. Similarly, testing often requires engineering resources, and the dynamic implications of this extension remain to be analyzed.

Second, future work can also focus on improving the mental models of managers working in multi-project development environments. The focus on mental models has proven successful in a number of other arenas with similar dynamics (Morecroft and Sterman 1994; Carroll *et al.* 1997). An important next step in this line of research is to use the model presented in this paper as the basis for a learning environment focused on improving managers' understanding of the dynamics surrounding resource allocation in product development.

Third, taking a cue from developments in the single-project domain—where SD models have evolved into full-scale decision support systems—the most ambitious direction for future work is to develop large-scale models of multi-project development environments that can help managers make resource allocation decisions on an ongoing basis. Such a dynamic resource planning system might take as inputs actual project experience and resource usage on downstream projects, and then generate realistic expectations for the completion of future projects. Developing such a tool represents an important opportunity for future uses of the SD method.

Acknowledgements

Work reported here was supported by the MIT Center for Innovation in Product Development under NSF Cooperative Agreement Number EEC-9529140, the

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Harley-Davidson Motor Company and the Ford Motor Company. Thanks to Drew Jones for providing useful comment on early versions of the model. The associate editor and two anonymous referees provided very helpful comments. Special thanks to Don Kieffer of Harley-Davidson for providing the catalyst to this study. An earlier version of this paper was titled 'Resource Dependence in Product Development Improvement Efforts.'

Appendix

To reduce the full dynamic system to a first-order map begin with the equation for *f(s)*, the fraction of task completed in the advanced phase in model year *s*:

$$f(s) = \frac{\int_0^T ACRR(t)\,dt}{A}.$$

Then, substitute in the definition for *ACRR*. This yields:

$$f(s) = \frac{\int_0^T ((1-P_A).\mathrm{Min}(DCR_A(t), \mathrm{Max}(NDC - CCWR(t), 0)))\,dt}{A}.$$

To reduce the model to a one-dimensional map requires two additional assumptions. First, some form must be specified for DCR_C and DCR_A, the desired completion rates. A mathematically convenient and plausible approach is to assume that these are calculated by allocating the remaining work stocks, A^r and C^r, evenly across the remaining in the model year. Thus:

$$DCR_A = A^r/(T-t)$$

$$DCR_C = C^r/(T-t).$$

Second, for simplicity, the rework and testing delays are eliminated. This can be accomplished by making the behavioral assumption that participants in the process know the defect rates, P_A and P_C, with certainty and plan their work accordingly. Specifically, the total work that will need to be accomplished on the current project over model year *t* is $C/(1-P_C)$, and the desired completion rate, less a capacity constraint, would be constant throughout the model year:

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

$$DCR_C = \frac{C}{1-P_C} \cdot \frac{1}{T}.$$

Substituting into the equations for *f(s)* yields:

$$f(s) = \frac{(1-P_A)}{A} \int_0^T \left(\mathrm{Min}\left(\frac{A}{1-P_A} \cdot \frac{1}{T}, \mathrm{Max}\left(NDC - \frac{C}{1-P_C} \cdot \frac{1}{T}, 0 \right) \right) \right) dt.$$

Then, recognizing that *C(s)*, the number of tasks remaining in the current phase, is simply equal to $A.(1-f(s-1))$ yields:

$$f(s) = \frac{(1-P_A)}{A} \int_0^T \left(\mathrm{Min}\left(\frac{A}{1-P_A} \cdot \frac{1}{T}, \mathrm{Max}\left(NDC - \frac{A(1-f(s-1))}{1-P_C} \cdot \frac{1}{T}, 0 \right) \right) \right) dt.$$

With this work allocation rule, each of the three elements is constant throughout the model year and, thus, the integration can be conducted separately and the expressions can be evaluated afterwards. Integrating the three separate elements of the Max and Min functions yields:

$$f(s) = \frac{(1-P_A)}{A} \cdot \mathrm{Min}\left(\frac{A}{1-P_A}, \mathrm{Max}\left(NDC \cdot T - \frac{A(1-f(s-1))}{1-P_A}, 0 \right) \right).$$

Substituting the definition for $P_D(s)$ again yields the following map for the fraction of advanced work completed each model year, *f(s)*:

$$f(s) = \mathrm{Min}\left(1, \mathrm{Max}\left(\frac{NDC \cdot T}{A/(1-P_A)} - \frac{1-P_A}{1-P_C}(1-f(s-1)), 0 \right) \right).$$

Finally, some algebra yields:

$$f(s) = \mathrm{Min}\left(1, \mathrm{Max}\left(\left(\frac{NDC \cdot T}{A/(1-P_A)} - \frac{1-P_A}{1-P_C} \right) + \frac{1-P_A}{1-P_C} \cdot f(s-1), 0 \right) \right).$$

Notes

1. Further, even in cases where it might be in the organization's best interest to abandon a project, the well-known and amply documented sunk cost fallacy suggests that managers will continue investing in projects well beyond the point of economic return.
2. The model is written using the VENSIM software produced by Ventanna Systems Inc. The fully documented model can be downloaded from http://web.mit.edu/nelsonr/www.

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

3. The model is simulated using the Euler integration method and runs in months with a time step of 0.25. In each case, it is run for 180 months to eliminate transients.

References

Abdel-Hamid T, Madnick S. 1991. *Software Project Dynamics: An Integrated Approach*. Prentice Hall: Englewood Cliffs, NJ.

Adler PS, Mandelbaum A, Nguyen V, Schwerer E. 1995. From product to process management: an empirically-based framework for analyzing product development time. *Management Science* **41**: 458–484.

Brehmer B. 1992. Dynamic decision making: human control of complex systems. *Acta Psychologica* **81**: 211–241.

Burchill G, Fine C. 1997. Time versus market orientation in product concept development: empirically-based theory generation. *Management Science* **43**: 465–478.

Carroll J, Sterman J, Markus A. 1997. Playing the maintenance game: how mental models drive organization decisions. In *Debating Rationality: Nonrational Elements of Organizational Decision Making*, Stern R, Halpern J (eds). ILR Press: Ithaca, NY.

Cooper KG. 1980. Navel ship production: a claim settled and a framework built. *Interfaces* **10**: 20–36.

Cooper RG. 1993. *Winning at New Products*. Addison Wesley: Reading, MA.

Cusumano M, Nobeoka K. 1998. *Thinking Beyond Lean*. Free Press: New York.

Dertouzos M, Lester R, Solow R. 1989. *Made in America: Regaining the Productive Edge*. MIT Press: Cambridge, MA.

Diehl E, Sterman JD. 1995. Effects of feedback complexity on dynamic decision making. *Organizational Behavior and Human Decision Processes* **62**: 198–215.

Ford DN, Sterman JD. 1998. Dynamic modeling of product development processes. *System Dynamics Review* **14**: 31–68.

Forrester JW. 1961. *Industrial Dynamics*. MIT Press: Cambridge, MA.

Funke J. 1991. Solving complex problems: exploration and control of complex systems. In *Complex Problem Solving: Principles and Mechanisms*, Sternberg R, Frensch P (eds). Lawrence Erlbaum Associates: Hillsdale, NJ.

Geuss, A de. 1988. Planning as learning. *Harvard Business Review* (March–April): 70–74.

Gupta A, Wilemon D. 1990. Accelerating the development of technology-based products. *California Management Review* (Winter): 24–44.

Homer J, Sterman J, Greenwood B, Perkola M. 1993. Delivery time reduction in pump and paper mill construction projects: a dynamics analysis of alternatives. *Proceedings of the 1993 International System Dynamics Conference*. Monterey Institute of Technology: Cancun, Mexico.

Jones AP. 1997. Sustaining process improvement in product development: the dynamics of part print mismatches. MS Thesis, Department of Civil and Environmental Engineering, Massachusetts Institute of Technology, Cambridge, MA.

Jones AP, Repenning NP. 1997. Sustaining Process Improvement at Harley-Davidson.

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Case study available from NP Repenning, MIT Sloan School of Management, Cambridge, MA 02142, USA.

Krahmer E, Oliva R. 1996. Improving Product Development Interval at AT&T Merrimack Valley Works. Case history available from authors, MIT Sloan School of Management, Cambridge, MA 02142, USA.

Morecroft J, Sterman J. 1994. *Modeling for Learning Organizations*. Productivity Press: Portland, OR.

Oliva R, Rockart S, Sterman J. 1998. Managing multiple improvement efforts. In *Management of Organizational Quality*, Feder DB, Goush S (eds). JAI Press: Stamford, CT; 1–55.

Paich M, Sterman J. 1993. Boom, bust, and failures to learn in experimental markets. *Management Science* **39**: 1439–1458.

Perlow L. 1999. The time famine. *Administrative Science Quarterly* **44**: 57–81.

Repenning NP. 1996. Reducing Product Development Time at Ford Electronics. Case history available from author, MIT Sloan School of Management, Cambridge, MA 02142.

Repenning NP. 1999a. Why Good Processes Produce Bad Results. Working Paper, Alfred P. Sloan School of Management, Cambridge, MA. http://web.mit.edu/nelsonr/www/.

Repenning NP. 1999b. A Simulation-based Approach to Understanding the Dynamics of Innovation Implementation. Alfred P. Sloan School of Management, Cambridge, MA. http://web.mit.edu/nelsonr/www/.

Repenning NP, Sterman JD. 2000. Getting quality the old-fashioned way: self-confirming attributions in the dynamics of process improvement. In *Improving Research in Total Quality Management*, Scott R, Cole R (eds). Sage: Newbury Park, CA.

Richardson GP, Pugh A. 1981. *Introduction to System Dynamics Modeling with DYNAMO*. MIT Press: Cambridge, MA.

Roberts EB. 1964. *The Dynamics of Research and Development*. Harper and Row: New York.

Roberts EB. 1978. *Managerial Applictions of System Dynamics*. Productivity Press: Cambridge, MA.

Sanderson S, Uzumeri M. 1997. *Managing Product Families*. Irwin: Chicago, IL.

Senge P. 1990. *The Fifth Discipline: The Art and Practice of the Learning Organizations*. Doubleday: New York.

Stata R. 1989. Organizational learning: the key to management innovation. *Sloan Management Review* **30**: 63–74.

Sterman JD. 1989a. Misperceptions of feedback in dynamic decision making. *Organizational Behavior and Human Decision Processes* **43**: 301–335.

Sterman JD. 1989b. Modeling managerial behavior: misperceptions of feedback in a dynamic decision making experiment. *Management Science* **35**: 321–339.

Sterman JD. 2000. *Business Dynamics: Systems Thinking and Modeling for a Complex World*. Irwin/McGraw Hill: Chicago, IL.

Sterman J, Repenning N, Kofman F. 1997. Unanticipated side effects of successful quality programs: exploring a paradox of organizational improvement. *Management Science*: 503–521.

Weil H, Bergan T, Roberts EB. 1978. The dynamics of R&D stragegy. In *Managerial Applications of System Dynamics*, Roberts EB (ed.). Productivity Press: Cambridge, MA.

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Wheelwright S, Clark K. 1992. *Revolutionizing Product Development: Quantum Leaps in Speed Efficiency and Quality*. The Free Press: New York.

Wheelwright S, Clark K. 1995. *Leading Product Development*. The Free Press: New York.

Ulrich K, Eppinger S. 1995. *Product Design and Development*. McGraw-Hill: New York.

Zangwill W. 1993. *Lightning Strategies for Innovation: How the World's Best Create New Projects*. Lexington Books: New York.

10991727, 2000, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/1099-1727(200023)16:3<173::AID-SDR196>3.0.CO;2-E by Bangladesh Hinari NPL, Wiley Online Library on [04/03/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License